

Du C au C++ moderne

Bloc 1 – Ce que C++ change immédiatement

Stéphane Abide Raphaël Granger

Université Côte d'Azur – LJAD

Cours : 1 heure – TD/TP : 2 heures

Séance 1 – Ce que C++ change immédiatement

Du C au C++ moderne

- Le C de MAM3 est un **prérequis**, pas le sujet du cours.
- Nous conservons le modèle impératif, mais nous renforçons les **types**, les **intentions** et la **gestion des ressources**.
- **À l'issue de ce bloc cours + TD/TP** : être prêt à comprendre pourquoi une classe protège la cohérence de ses données et maîtrise leur durée de vie.

Connu en C $\longrightarrow$ différence C++ $\longrightarrow$ choix moderne

Même syntaxe $\neq$ même langage

```
#include <stdlib.h>

int main(void)
{
    int n = 10;
    int* data = malloc(n * sizeof *data);

    if (data == NULL)
        return 1;

    free(data);
}
```

Information utile

```
void* malloc(size_t size);
```

malloc renvoie donc un void*.

Question. Ce programme C est-il aussi un programme C++ valide ?

Démonstration live

```
gcc -std=c17 -Wall -Wextra -pedantic \
    c_vs_cpp.c -o c_vs_cpp_c

g++ -std=c++20 -Wall -Wextra -pedantic \
    c_vs_cpp.cpp -o c_vs_cpp_cpp
```

Même syntaxe $\neq$ même langage

```
#include <stdlib.h>

int main(void)
{
    int n = 10;
    int* data = malloc(n * sizeof *data);

    if (data == NULL)
        return 1;

    free(data);
}
```

Information utile

```
void* malloc(size_t size);
```

malloc renvoie donc un void*.

Question. Ce programme C est-il aussi un programme C++ valide ?

Démonstration live

```
gcc -std=c17 -Wall -Wextra -pedantic \
    c_vs_cpp.c -o c_vs_cpp_c

g++ -std=c++20 -Wall -Wextra -pedantic \
    c_vs_cpp.cpp -o c_vs_cpp_cpp
```

Après la compilation

malloc renvoie un void*. En C, ce void* peut ici être converti implicitement en int*. En C++, cette conversion implicite n'est pas permise.

Un programme C correct n'est pas nécessairement un programme C++ correct.

C++ est plus strict sur certaines conversions implicites de types.

On pourrait forcer explicitement la conversion, mais ce code resterait écrit dans un style C.

Le compilateur comme partenaire

```
g++ -std=c++20 -Wall -Wextra -pedantic main.cpp -o main
```

g++	compilation d'un programme C++
-std=c++20	standard explicitement choisi
-Wall -Wextra	diagnostics supplémentaires
-pedantic	aide à rester proche du standard

Règle de travail

Compiler tôt, avec les avertissements, et traiter les diagnostics comme des informations sur le programme.

Une erreur de compilation peut révéler que le code ne correspond pas à l'intention exprimée par les types.

Entrées-sorties : des fonctions aux flux typés

Acquis en C

```
double x;

printf("x ? ");
scanf("%lf", &x);

printf("x^2 = %.3f\n", x*x);
```

Choix C++

```
double x{};

std::cout << "x ? ";
std::cin >> x;

std::cout << "x^2 = "
          << x*x << '\n';
```

- `std::cout` et `std::cin` sont les **flux standard** de sortie et d'entrée.
- `std::` indique que ces noms appartiennent à la **bibliothèque standard C++**.
- Avec `<<` et `>>`, le **type de la donnée détermine l'opération** : plus de `%d`, `%f` ou `%lf` à accorder manuellement.

Message clé : en C++, le type pilote l'opération

```
int n{3};
double x{2.5};

std::cout << n << " " << x << '\n';
```

`std::cout` utilise directement le type : `<<` sait écrire successivement un `int` puis un `double`.

Nous verrons plus tard comment C++ permet à un même opérateur d'avoir plusieurs comportements selon les types.

Initialiser dès la création

Une variable doit avoir un état initial défini.

Donner immédiatement la première valeur

```
int iterations{100};  
double tolerance{1e-8};  
double residual{};           // 0.0  
int i = 3.9;                 // autorise  
                             // i vaut 3  
int j{3.9};                  // erreur :  
                             // conversion reductrice
```

- {} fournit une syntaxe d'initialisation uniforme.
- Les accolades permettent aussi de détecter certaines conversions réductrices.
- T x{}; permet une initialisation à zéro.

Question. Dans un calcul scientifique, préférez-vous une erreur de compilation ou une troncature silencieuse ?

Déclarer, initialiser, affecter

```
double x;                    // déclaration  
                             // valeur indeterminée  
double y{2.0};               // déclaration +  
                             // initialisation  
y = 3.0;                     // affectation
```

Initialiser n'est pas affecter

Initialiser fixe la première valeur de l'objet.
Affecter modifie un objet qui existe déjà.

const : une valeur qui ne doit pas changer

```
const int dimension{3};  
const double tolerance{1e-10};  
  
double residual{1.0};  
residual = 0.25;      // normal: la valeur evolue  
tolerance = 1e-6;     // erreur: tolerance est const
```

- const indique qu'une valeur ne doit pas être modifiée.
- Le compilateur vérifie cette règle.
- Cela évite certaines modifications accidentelles.

Une règle que le compilateur peut vérifier

Exprimer nos intentions dans les types permet au compilateur de détecter certaines erreurs.

Invariant : une propriété du programme qui doit rester vraie. Nous en verrons des exemples plus intéressants avec les classes.

Modifier l'objet de l'appelant : pointeur en C, référence en C++

Acquis en C

```
void swap_int(int* a, int* b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

swap_int(&x, &y);
```

Choix C++

```
void swap_int(int& a, int& b)
{
    int tmp{a};
    a = b;
    b = tmp;
}

swap_int(x, y);
```

C : pointeur

l'appel transmet une adresse
déréférencement `*a`
`int*` peut représenter un pointeur nul

C++ : référence

l'appel reste `swap_int(x, y)`
utilisation directe de `a`
`int&` exprime ici qu'un objet est attendu

Important

C++ ne supprime pas les pointeurs : pointeurs et références expriment des situations différentes. La référence indique que la fonction agit directement sur l'objet de l'appelant.

Une référence est un alias

```
double x{2.0};
double& alias{x};

alias *= 3.0;           // x vaut maintenant 6.0

std::cout << &x << '\n';
std::cout << &alias << '\n';
```

Question. Qu'observe-t-on ?

Référence

- autre nom pour un objet ;
- doit être liée à un objet ;
- ne change pas d'objet ;
- s'utilise directement comme l'objet.

Référence : un objet est nécessaire.
avoir un sens.

```
double& bad;
// erreur : doit etre liee

double& also_bad{2.0};
// erreur : reference non const
// vers un temporaire
```

Même objet

x et **alias** désignent le même objet :
modifier l'un ou prendre son adresse
revient à agir sur l'autre.

Pointeur

- contient une adresse ;
- peut valoir `nullptr` ;
- peut changer de cible ;
- doit être déréférencé pour accéder à l'objet.

Pointeur : l'absence ou le changement de cible peut

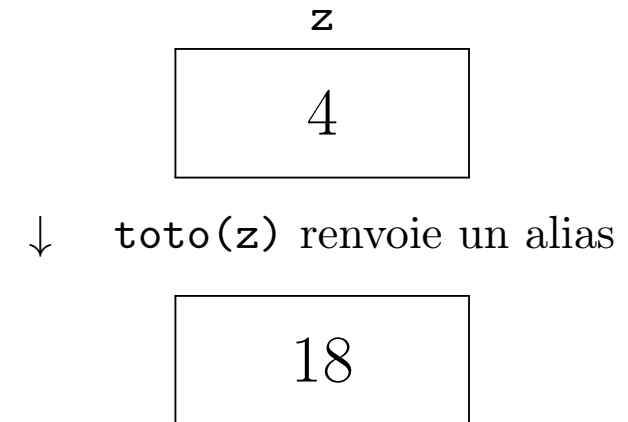
Renvoyer une référence

```
int& toto(int& value)
{
    return value;
}

int z{4};

toto(z) = 18;

std::cout << z << '\n'; // 18
```



Question. Quelle valeur est réellement modifiée ?

À lire dans la signature

Une référence peut être un paramètre, mais aussi un résultat. L'affectation à `toto(z)` agit donc directement sur `z`.

Une fonction ne doit pas renvoyer une référence vers une variable locale qui disparaît à la fin de l'appel.

Même objet

Le type de retour `int&` annonce que la fonction ne fabrique pas un nouvel entier : elle donne un autre accès à `z`.

La signature indique comment l'argument est utilisé

T – copie

- la fonction reçoit une copie ;
- petit type ou copie voulue.

```
double square(double x)
{
    return x * x;
}
```

T& – modification

- la fonction agit sur l'objet original ;
- à choisir lorsqu'elle doit le modifier.

```
void normalize(Vector& v);
```

const T& – lecture

- la fonction lit l'objet original sans le modifier ;
- à choisir lorsque l'on ne souhaite pas le copier.

```
double norm(const Vector& v);
```

Ne pas appliquer mécaniquement const T&

Pour `int`, `double` ou `char`, le passage par valeur est généralement naturel. Pour un objet plus gros que l'on veut seulement lire, `const T&` évite une copie.

En lisant une signature, on doit pouvoir savoir : copie, modification ou simple lecture.

Quelle signature choisir ?

```
struct Point {  
    double x;  
    double y;  
};  
  
double norm( /* ? */ );  
void translate( /* ? */,  
               double dx, double dy );  
Point midpoint( /* ? */, /* ? */ );
```

Questions à poser avant d'écrire le type

- ❶ La fonction doit-elle modifier l'argument ?
- ❷ Une copie est-elle souhaitée ?
- ❸ Ne fait-elle qu'observer l'objet ?

Quelle signature choisir ?

```
struct Point {  
    double x;  
    double y;  
};  
  
double norm( /* ? */ );  
void translate( /* ? */,  
               double dx, double dy );  
Point midpoint( /* ? */, /* ? */ );
```

Questions à poser avant d'écrire le type

- ❶ La fonction doit-elle modifier l'argument ?
- ❷ Une copie est-elle souhaitée ?
- ❸ Ne fait-elle qu'observer l'objet ?

Signatures retenues

```
double norm(const Point& p);
```

```
void translate(Point& p, double dx, double dy);
```

```
Point midpoint(const Point& a, const Point& b);
```

Pour midpoint, les entrées sont observées par référence constante ; le résultat est un nouvel objet renvoyé par valeur.

Le diagnostic fait partie du cours

Question. Pourquoi ce code ne compile-t-il pas ? Quelle signature révèle l'intention réelle ?

```
void translate(const Point& p, double dx, double dy) {  
    p.x += dx;           // error: assignment in read-only object  
    p.y += dy;  
}
```

Correction

```
void translate(Point& p, double dx, double dy);
```

Le diagnostic révèle que la signature dit « lecture seule », alors que le code essaie de modifier p.

Fil rouge : le tableau dynamique hérité du C

Acquis en C

```
typedef struct {  
    int capacity;  
    int size;  
    int *array;  
} MyArray;  
  
MyArray *create_array(int n);  
void push_back(MyArray *, int);  
void destroy_array(MyArray *);
```

Choix C++

```
struct IntArray {  
    int capacity{};  
    int size{};  
    int* data{nullptr};  
};  
  
void create_array(IntArray& array,  
                 int initial_capacity);  
void push_back(IntArray& array, int value);  
void destroy_array(IntArray& array);
```

Même problème, mais les types expriment maintenant plus clairement les intentions. La responsabilité de la mémoire reste encore manuelle.

Séance 1 – Bilan

- ① Pourquoi `int n{3.9};` est-il refusé ?
- ② Une fonction observe un grand objet : `T`, `T&` ou `const T&` ?
- ③ Une fonction doit modifier son argument : quelle signature ?
- ④ Quand un pointeur reste-t-il plus juste qu'une référence ?

À maîtriser avant la suite

Lire une signature permet de savoir : copie, modification, lecture seule ou absence possible. Les intentions sont mieux exprimées, mais notre `IntArray` gère encore sa mémoire manuellement.